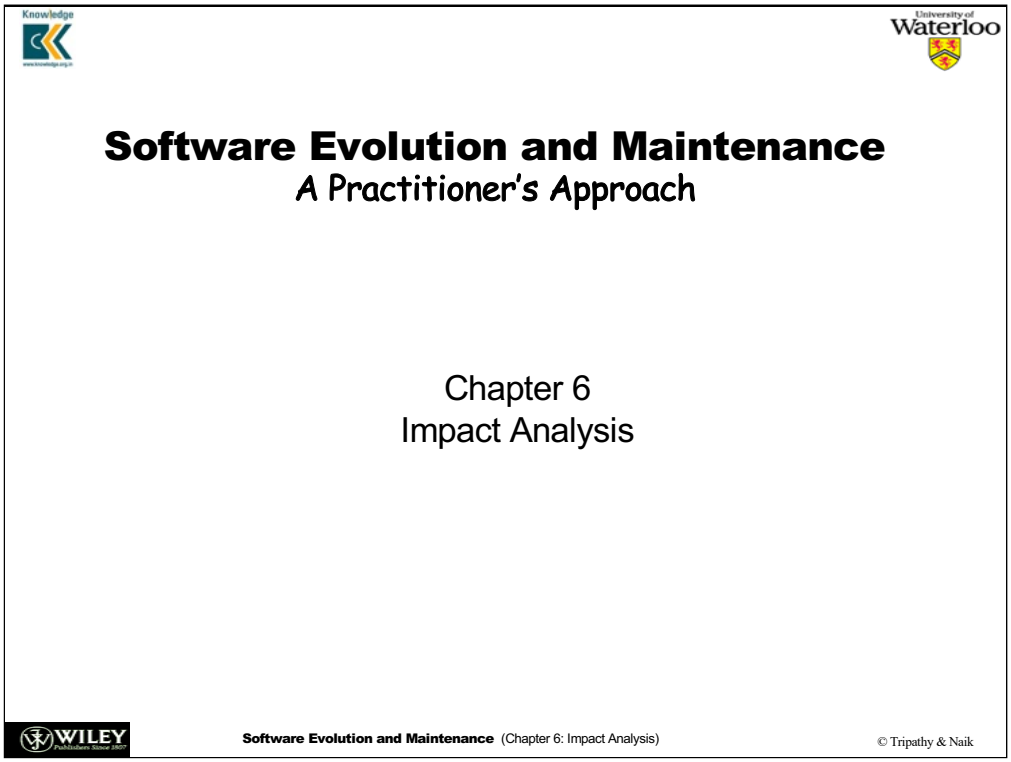

Knowledge
University of Waterloo
Software Evolution and Maintenance
A Practitioner's Approach
Chapter 6
Impact Analysis
WILEY
Software Evolution and Maintenance (Chapter 6: Impact Analysis)
© Tripathy & Naik

Outline of the Chapter

6.1 General Idea
6.2 Impact Analysis Process
6.2.1 Identifying the SIS
6.2.2 Analysis of Traceability Graph
6.2.3 Identifying the Candidate Impact Set
6.3 Dependency-based Impact Analysis
6.3.1 Call Graph
6.3.2 Program Dependency Graph
6.4 Ripple Effect
6.4.1 Computing Ripple Effect
6.5 Change Propagation Model
6.5.1 Recall and Precision of Change Propagation Heuristics
6.5.2 Heuristics for Change Propagation
6.5.3 Empirical Studies

© Tripathy & Naik

6.1 General Idea

- The maintenance process is started by performing impact analysis.
- Impact analysis basically means identifying the components that are impacted by the Change Request (CR).
- Impact of the changes are analyzed for the following reasons:
  - to estimate the cost of executing the change request.
  - to determine whether some critical portions of the system are going to be impacted due to the requested change.
  - to record the history of change related information for future evaluation of changes.
  - to understand how items of change are related to the structure of the software.
  - to determine the portions of the software that need to be subjected to regression testing after a change is effected.

© Tripathy & Naik

6.1 General Idea

- Implementation of change requests impact all kinds of **artifacts**, including the source code, requirements, design documentation, and test scenarios.
- Therefore, impact analysis traceability information can be used in performing impact analysis.
- Gotel and Finkkelstein define traceability as the ability to describe and follow the life of an artifact in both the forward and backward directions.
- Bohner and Arnold define traceability as the ability to trace between software artifacts generated and modified during the software product life cycle.
- Thus, **traceability** helps software developers understand the relationships among all the software artifacts in a project.
- There are two broad kinds of traceability:

  (i) **horizontal** (external) traceability.

  (ii) **vertical** (internal) traceability.
- **Traceability** of artifacts between different models is known as external traceability, whereas internal traceability refers to tracing dependent artifacts within the same model.

© Tripathy & Naik

6.1 General Idea

- A topic related to impact analysis is ripple effect analysis.
- Ripple effect means that a modification to a single variable may require several parts of the software system to be modified.
- Analysis of ripple effect reveals what and where changes are occurring.
- Measurement of ripple effects can provide the following information about an evolving software systems:

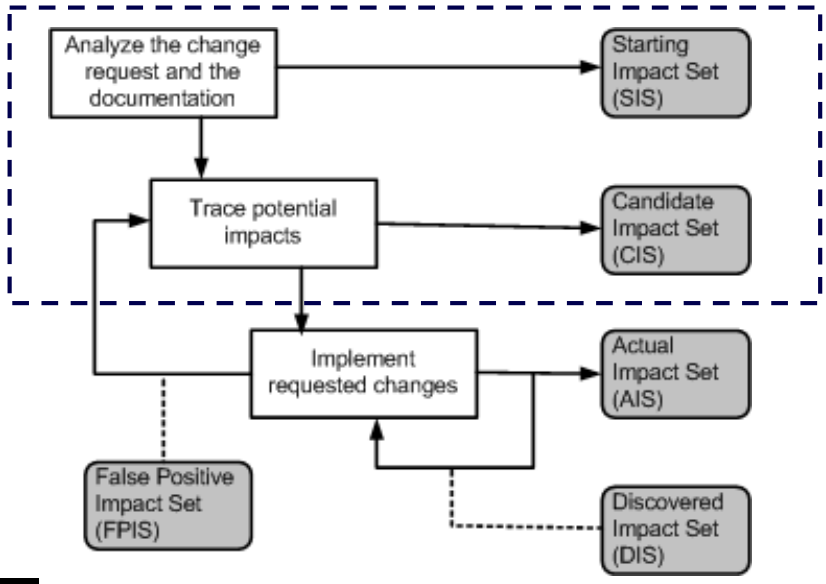
  (i) between successive versions of the same system, measurement of ripple effect will tell us how the software's complexity has changed.

  (ii) when a new module is added to the system, measurement of ripple effect on the system will tell us how the software's complexity has changed because of the addition of the new module.

© Tripathy & Naik

6.2 Impact Analysis Process

Figure 6.1 Impact analysis process ©IEEE, 2008



© Tripathy & Naik

6.2 Impact Analysis Process

- **Starting Impact Set (SIS):** The initial set of objects (or components) presumed to be directly impacted by a software CR is called SIS.
  - The set of components to be modified to implement the CR
- **Candidate Impact Set (CIS):** The full set of objects (or components) estimated to be impacted according to a certain impact analysis approach is called CIS.
  - The SIS is augmented with the components that are estimated to change as a consequence of changes to the components in the SIS
- The impact analysis process ends with the identification of the CIS.
  - Decisions on whether it is worth implementing the change request and how many resources to assign depends on the results of the impact analysis.
  - It is important to make impact analysis as more accurate as possible …

© Tripathy & Naik

6.2 Impact Analysis Process

- After the change is (designed and) implemented the set of actually changed components emerges
- **Actual Impact Set (AIS):** The set of objects (or components) actually changed as a result of implementing a CR.
  - The goal of the impact analysis process is to minimize the differences between AIS and CIS
- Two sets of components can be computed as difference between the CIS and AIS
- **Discovered Impact Set (DIS):** The set of new objects (or components), not contained in CIS, discovered to be impacted while implementing a CR (DIS = AIS \ CIS)
- **False Positive Impact Set (FPIS):** The set of objects (or components) estimated to be impacted by an implementation of a CR but not actually impacted by the CR (FPIS = CIS \ AIS).
  - where \ denotes set difference.

© Tripathy & Naik

6.2 Impact Analysis Process

Two traditional information retrieval metrics can be used to evaluate the quality (accuracy) of the Impact Analysis Process:

- **Recall:** It represents the fraction of actual impacts contained in CIS, and it is computed as the ratio of |CIS ∩ AIS| to |AIS|.
  - The value of recall is 1 when DIS is empty.
- **Precision:** It represents the fraction of candidate impacts that are actually impacted, and it is computed as the ratio of |CIS ∩ AIS| to |CIS|.
  - For an empty FPIS set, the value of precision is 1.
- Note that if AIS is equal to CIS, both **recall** and **precision** are computed to be equal to 1.

Other specific metrics for impact analysis have also been proposed, namely **Adequacy** and **Effectiveness**

6.2 Impact Analysis Process

Adequacy

- Adequacy of an impact analysis approach is the ability of the approach to identify all the affected elements to be modified. Ideally, AIS⊆CIS.
- Adequacy is expressed in terms of a performance metric called inclusiveness, as follows:

$$Inclusiveness = \begin{cases} 1 & \text{if AIS} \subseteq \text{CIS} \\ 0 & \text{otherwise} \end{cases}$$

- The concept of adequacy is essential to assessing the quality of an impact analysis approach.
- An inadequate approach is in fact useless, as it provides the maintenance engineer with incorrect information.
- For example, if inclusiveness is 0, the approach cannot be used because it does not provide the maintenance engineer with all the components to be analyzed.

© Tripathy & Naik

6.2 Impact Analysis Process

Effectiveness

- The ability of an impact analysis technique to generate results, that actually benefit the maintenance tasks, is known as its effectiveness.
- Effectiveness is expressed in terms of three fine grained characteristics as follows:
  - Ripple-sensitivity
  - Sharpness
  - Adherence

© Tripathy & Naik

6.2 Impact Analysis Process

Effectiveness

Ripple-sensitivity

- **Ripple-sensitivity** implies producing results that are influenced by ripple effect.

- The set of objects that are directly affected by the change is denoted by DISO (directly impacted set of objects).
- Similarly, the set of objects that are indirectly impacted by the change is denoted by IISO (indirectly impacted set of objects).
- The cardinality of IISO is an indicator of ripple effect.
- The software maintenance personnel expect that the cardinality of IISO is not far from the cardinality of DISO.
- Therefore, the concept of Amplification, as defined below, is used as a measure of Ripple-sensitivity.

  Amplification = | IISO | / | DISO | → 1; where | . | denotes the cardinality operator.

© Tripathy & Naik

6.2 Impact Analysis Process

Effectiveness

Sharpness

- **Sharpness** is the ability of an impact analysis approach to avoid having to include objects in the CIS that are not needed to be changed.
- Sharpness is expressed by means of Change Rate as defined below:

  ChangeRate = | CIS | / | System |
- *ChangeRate* falls in the range from 0 to 1.

Adherence

- **Adherence** is the ability of the approach to produce a CIS which is as close to AIS as possible.
- A small difference between CIS and AIS means that a small number of candidate objects fail to be included in the actual modification set.
- Adherence is expressed by S-Ratio as follows:

  S-Ratio = | AIS | / | CIS |
- *S-Ratio* takes on values in the range from 0 to 1.

© Tripathy & Naik

6.2.1 Identifying the Starting Impact Set (SIS)

- Impact analysis begins with identifying the SIS.
- The larger the system the higher the effort required to identify the SIS
- The CR specification, documentation, and source code are analyzed to find the SIS.
- Mapping the concepts defined in a change request specification onto source code components is a difficult task
  - Concept assignment: "discovering human-oriented concepts and assigning them to their realizations"
  - This process cannot be completely automated, as concepts and programs are not at the same level of abstraction, , thereby necessitating human interactions

© Tripathy & Naik

15

Approaches to Concept Assignment

- Static analysis
- Dynamic analysis
- Use of informal information, e.g., the meaning of the program identifiers
- Natural language processing and Information Retrieval techniques
  - Identifiers in source code, documentation, CVS comments and change logs
- Combination of different approaches

6.2.1 Identifying the Starting Impact Set (SIS)

- There are several methods to identify concepts, or features, in source code.
  - Static analysis
  - Dynamic analysis
  - Use of informal information, e.g., the concepts behind the program identifiers
  - Natural language processing and Information Retrieval techniques
    - Identifiers in source code, documentation, CVS comments and change logs
  - Combination of different approaches

© Tripathy & Naik

6.2.1 Identifying the Starting Impact Set (SIS)

- As an example of approach based on **static analysis**, Chen and Rajlich proposed a dependency graph based feature location method for C programs.
- The component dependency graph is searched, generally beginning at the main().
- Functions are chosen one at a time for a visit.
- The maintenance personnel reads the documentation, code, and dependency graph to comprehend the component before deciding if the component is related to the feature under consideration.

© Tripathy & Naik

6.2.1 Identifying the Starting Impact Set (SIS)

- As an example of approach based on **dynamic analys**, the software reconnaissance methodology proposed by Wilde and Scully is based on the idea that some programming concepts are selectable, because their execution depends on a specific input sequence.
- Selectable program concepts are known as features. By executing a program twice, one can often find the source code implementing the features:

  (i) execute the program once with a feature and once without the feature.

  (ii) mark portions of the source code that were executed the first time but not the second time.

  (iii) the marked code are likely to be in or close to the code implementing the feature.

© Tripathy & Naik

6.2.1 Identifying the Starting Impact Set (SIS)

- Concerning the **use of the concepts behind the program identifiers**, the "grep" pattern matching utility available on most Unix systems and similar search tools are commonly used by programmers.
- However, the tool has some deficiencies:
  - it is based on the correspondence between the name for the concept assigned by the programmer and an identifier in the code.
- The technique often fails when the concepts are hidden in the source code, or when the programmer fails to guess the program identifiers.

© Tripathy & Naik

6.2.1 Identifying the Starting Impact Set (SIS)

- **Natural language processing and Information Retrieval techniques** can also be used.
- In this case the change request is used as a query to search the document space composed by the source code and documentation components. More later …
- If requirements analysis documentation and traceability is available, then the change request can be queried on a restricted document space composed on requirements and uses cases and these can then be traced onto the other software artifacts.

© Tripathy & Naik

6.2.2 Analysis of Traceability Graph

- The graph shows the horizontal traceability of the system.
- The graph that is so constructed reveals the relationships among work products.

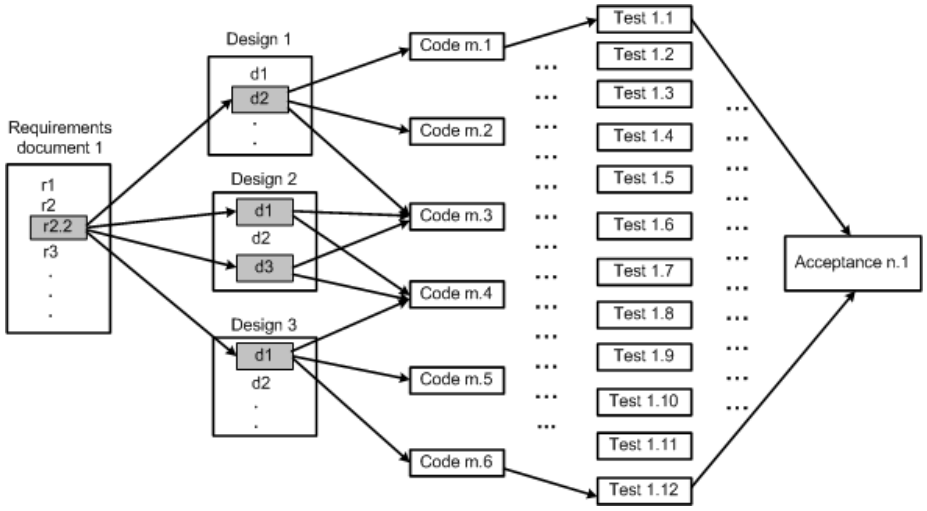


Figure 6.2 Traceability in software work products ©IEEE, 1991

22

Vertical vs Horizontal Traceability

- Vertical traceability: the ability to trace dependent artefacts within a model
  - Most approaches focused on source code artefacts
    - Program slicing to identify dependencies among statements
    - Source code analysis and reverse engineering to identify dependencies among source code modules
  - Other approaches focused on software artefacts at higher abstraction levels, such requirements, or UML models
- Horizontal traceability: the ability to trace artefacts between different models
  - Requirements, design models, source code, test cases, defect reports, rationale elements

23

Vertical vs Horizontal Traceability (2)

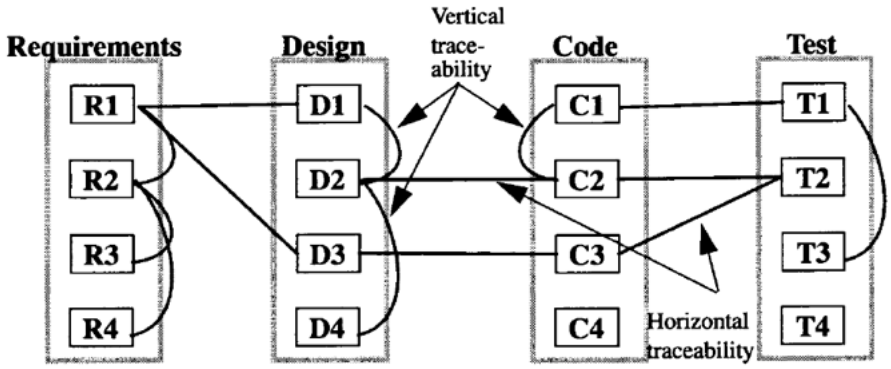


Figure taken from Lindvall and Sandahl (JSM 1998)
Original classification by Pfleeger and Bohner (CSM 1990)

Horizontal traceability is mainly used during SIS identification.
CIS identification also requires vertical traceability ...

6.2.2 Analysis of Traceability Graph

- The graph has four categories of nodes: requirements, design, code, and test.
- The edges within a silo represent vertical traceability for the kind of work product represented by the silo.

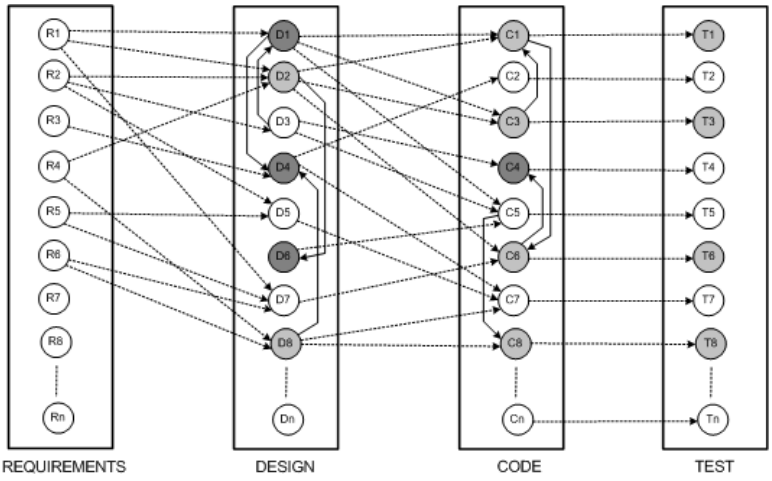


Figure 6.3 Underlying graph for maintenance ©IEEE, 1991

6.2.2 Analysis of Traceability Graph

- If some changes are made to requirement object "R4," the results of horizontal traceability and vertical traceability are shown in Figure 6.4.
- The horizontally traced objects have been shown as lightly shaded circles, whereas the vertically traced objects have darkly shaded circles.

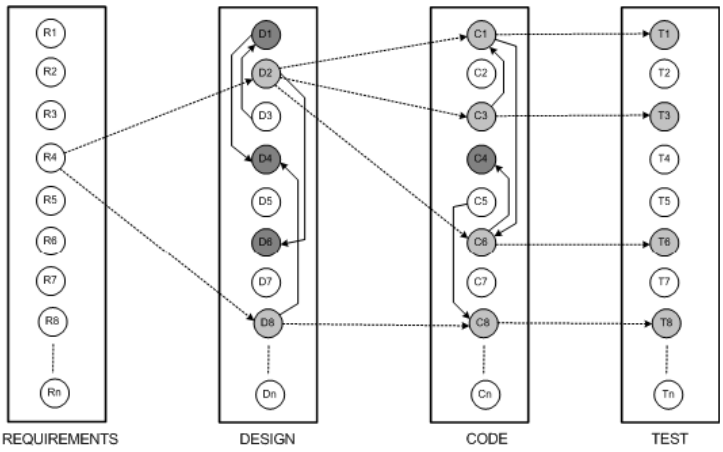


Figure 6.4 Determine work product impact ©IEEE, 1991

© Tripathy & Naik

26

Traceability dimensions

Dimension	Cathegories	
D_1	Vertical	Horizonthal
D_2	Structural	Knowledge based
D_3	Implicit	Explicit

Adapted from: Bianchi et al. (IWPC, 2000)

27

Structural vs Knowledge-based Traceability

- Structural traceability: derived by parsing techniques
  - The artefacts are defined in a formal language
    - Not only source code, but also high level models expressed in a formal (e.g., XML) and/or graphical language (e.g., UML)
- Knowledge-based traceability: deals with informal information and requires human interaction
  - Heuristics + knowledge of the application/solution domain
    - Name tracing rules, dynamic analysis rules, logic based rules
    - Information retrieval and version history mining methods
  - Discovered links are validated to avoid false positives
  - Links may be missed (artefacts will not be included in CIS)

28

Implicit vs Explicit Traceability

- Explicit Traceability: Links are stored in some repository
  - Required when the recovery process is expensive or requires human interaction
- Implicit Traceability: Links are derived on-the-fly from software artefacts when needed
  - Used when the recovery process is fast and reliable
- Hybrid approaches: some type of links are stored explicitly while other types of links are recovered when needed

6.2.3 Identifying the Candidate Impact Set

- A CIS is identified in the next step of the impact analysis process.
- The SIS is augmented with software lifecycle objects (SLOs) that are likely to change because of changes in the elements of the SIS.
- Changes in one part of the software system may have direct impacts or indirect impacts on other parts.
- Both direct impact and indirect impact are explained in the following.
  - **Direct impact:** A direct impact relation exists between two entities, if the two entities are related by a fan-in and/or fan-out relation.
  - **Indirect impact:** If an entity A directly impacts another entity B and B directly impacts a third entity C, then we can say that A indirectly impacts C.

© Tripathy & Naik

6.2.3 Identifying the Candidate Impact Set

- let us consider the directed graph in Figure 6.5 with ten SLOs.
- Each SLO represents a software artifact connected to other artifacts.
- Dependencies among SLOs are represented by arrows.
- In the figure, SLO1 has an indirect impact from SLO8 and a direct impact from SLO9.
- The in-degree of a node i reflects the number of known nodes that depend on i.

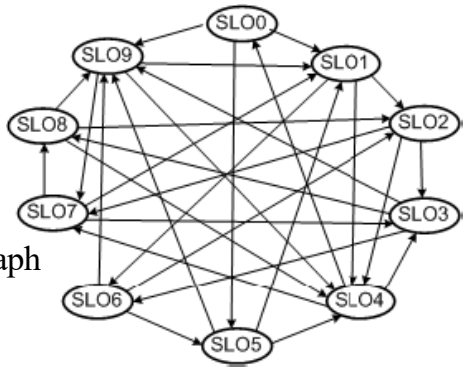


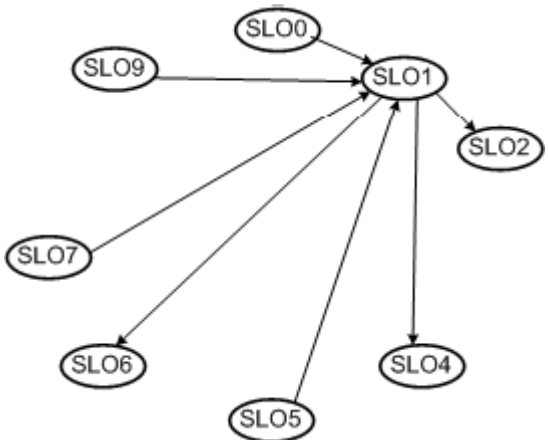
Figure 6.5 Simple directed graph of SLOs ©IEEE, 2002

© Tripathy & Naik

6.2.3 Identifying the Candidate Impact Set

- Figure 6.6 shows the four nodes – SLO0, SLO5, SLO7 and SLO9 – that are dependent on SLO1, and the in-degree of SLO1 is four.
- In addition, the out-degree of SLO1 is three.

Figure 6.6 In-degree and out-degree of SLO1 ©IEEE, 2002



© Tripathy & Naik

6.2.3 Identifying the Candidate Impact Set

- The connectivity matrix of Table 6.1 is constructed by considering the SLOs and the relationships shown in Figure 6.5.
- A reachability graph can be easily obtained from a connectivity matrix.
- A reachability graph shows the entities that can be impacted by a modification to a SLO, and there is a likelihood of over-estimation.

	SLO0	SLO1	SLO2	SLO3	SLO4	SLO5	SLO6	SLO7	SLO8	SLO9
SLO0		x				x				x
SLO1			x		x		x			
SLO2				x	x			x		
SLO3							x		x	x
SLO4	x			x				x		
SLO5		x			x					x
SLO6			x			x				x
SLO7		x		x					x	
SLO8			x		x					x
SLO9		x			x			x		

Table 6.1: Relationships represented by a connectivity matrix [13] (©[2002] IEEE).

6.2.3 Identifying the Candidate Impact Set

- The dense reachability matrix of Table 6.2 has the risk of over-estimating the CIS.
- To minimize the occurrences of false positives, one might consider the following two approaches:
  - Distance based approach.
  - Incremental approach.

	SLO0	SLO1	SLO2	SLO3	SLO4	SLO5	SLO6	SLO7	SLO8	SLO9
SLO0		x	x	x	x	x	x	x	x	x
SLO1	x		x	x	x	x	x	x	x	x
SLO2	x	x		x	x	x	x	x	x	x
SLO3	x	x	x		x	x	x	x	x	x
SLO4	x	x	x	x		x	x	x	x	x
SLO5	x	x	x	x	x		x	x	x	x
SLO6	x	x	x	x	x	x		x	x	x
SLO7	x	x	x	x	x	x	x		x	x
SLO8	x	x	x	x	x	x	x	x		x
SLO9	x	x	x	x	x	x	x	x	x	

Table 6.2: Relationships represented by a reachability matrix [13] (©[2002] IEEE)

6.2.3 Identifying the Candidate Impact Set

- **Distance based approach:** In this approach, SLOs which are farther than a threshold distance from SLO i are not be considered to be impacted by changes in SLO i.
- **Incremental approach:** In this approach, the CIS is incrementally constructed. For every SLO in the SIS, one considers all the SLOs interacting with it, and only SLOs that are actually impacted by the change request are put in the CIS.

	SLO 0	SLO 1	SLO 2	SLO 3	SLO 4	SLO 5	SLO 6	SLO 7	SLO 8	SLO 9
SLO0		1	2	3	2	1	2	2	3	1
SLO1	2		1	2	1	2	1	2	3	2
SLO2	2	2		1	1	3	2	1	2	2
SLO3	3	2	2		2	2	1	2	1	1
SLO4	1	2	3	1		2	2	1	2	2
SLO5	2	1	2	2	1		2	2	3	1
SLO6	3	2	1	2	2	1		2	3	1
SLO7	3	1	2	1	2	3	2		1	2
SLO8	3	2	1	2	1	3	3	2		1
SLO9	2	1	2	2	1	3	2	1	2	

Table 6.3: Relationship with distance indicators [13] (©[2002] IEEE).

© Tripathy & Naik

6.3 Dependency-based Impact Analysis

- In general, source code objects are analyzed to obtain vertical traceability information.
- Dependency based impact analysis techniques identify the impact of changes by analyzing syntactic dependencies, because syntactic dependencies are likely to cause semantic dependencies.
- Two traditional impact analysis techniques are explained:
  - The first technique is based on call graph.
  - the second one is based on dependency graph.

© Tripathy & Naik

6.3.1 Call Graph

- A call graph is a directed graph in which a node represents a function, a component, or a method.
- An edge between two nodes A and B means that A may invoke B
- Programmers use call graphs to understand the potential impacts that a software change may have.
- An example call graph has been shown in Figure 6.7

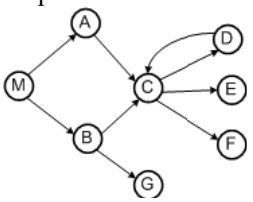


Figure 6.7 Example of a call graph ©IEEE, 2003

- Let P be a program, G be the call graph obtained from P, and p be some procedure in P
- A key assumption in the call graph-based technique is that some change in p has the potential to impact changes in all nodes reachable from p in G.

© Tripathy & Naik

6.3.1 Call Graph

- The call graph-based approach to impact analysis suffers from many disadvantages as follows:
  - impact analysis based on call graphs can produce an imprecise impact set. For example, in Figure 6.7, one cannot determine the conditions that cause impacts of changes to propagate from M to other procedures.
  - impact propagations due to procedure returns are not captured in the call graph-based technique. Suppose that, in Figure, *E* is modified and control returns to C.. Now, following the return to C, it cannot be inferred whether impacts of changing E propagates into none, both, A, or B.

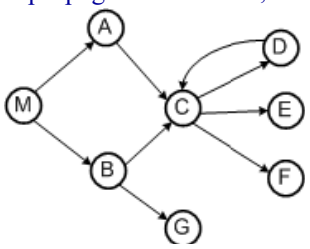


Figure 6.7 Impact analysis process ©IEEE, 2003

6.3.1 Call Graph

- Let us consider an execution trace as shown in below.

  M B r A C D r E r r r r x. Where r and x represent function returns and program exits.
- The impact of the modification of M with respect to the given trace is computed by forward searching in the trace to find:
  - procedures that are indirectly or directly invoked by E; and
  - procedures that are invoked after E terminates.
- One can identify the procedures into which E returns by performing backward search in the given trace.
- For example, in the given trace, E does not invoke other entities, but it returns into M, A, and C.
- Due to a modification in E, the set of potentially impacted procedures is {M,A,C, E}.

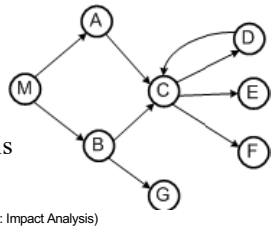


Figure 6.7 Impact analysis process ©IEEE, 2003

© Tripathy & Naik

6.3.2 Program Dependency Graph

- In the program dependency graph (PDG) of a program:
  (i) each simple statement is represented by a node, also called a vertex;
  (ii) each predicate expression is represented by a node.
- There are two types of edges in a PDG: data dependency edges and control dependency edges.
- Let v_i and v_j be two nodes in a PDG.
- If there is a data dependency edge from node v_i to node v_j , then the computations performed at node v_i are directly dependent upon the results of computations performed at node v_j .
- A control dependency edge from node v_i to node v_j indicates that node v_i may execute based on the result of evaluation of a condition at v_j .

© Tripathy & Naik

6.3.2 Program Dependency Graph

- Figure 6.10shows the PDG of the program given in Figure 6.9.
- Data dependencies are shown as solid edges, whereas control dependencies are shown as dashed edges.

Figure 6.9 Example program ©ACM, 1990

```
    begin
S1 :    read(X)
S2 :    if(X < 0)
        then
S3 :        Y = f1(X);
S4 :        Z = g1(X);
        else
S5 :        if(X = 0)
            then
S6 :            Y = f2(X);
S7 :            Z = g2(X);
            else
S8 :            Y = f3(X);
S9 :            Z = g3(X);
            end_if;
        end_if;
S10 :   write(Y);
S11 :   write(Z);
```

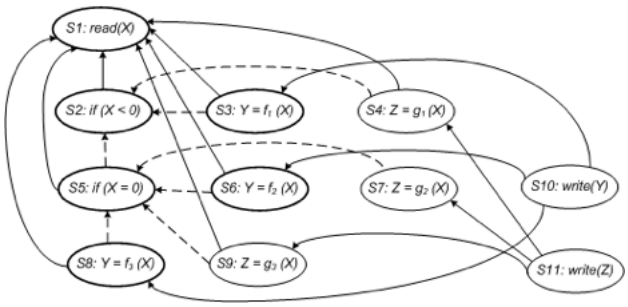


Figure 6.10 Program dependency graph of the program in Figure 6.9

© Tripathy & Naik

6.3.2 Program Dependency Graph

Static Program Slice

- A static program slice is identified from a PDG as follows:
  - for a variable var at node n, identify all reaching definitions of var.
  - find all nodes in the PDG which are reachable from those nodes.
  - The visited nodes in the traversal process constitute the desired slice.
- Consider the program in the previous slide and variable Y at S10.
- First, find all the reaching definitions of Y at node S10 – and the answer is the set of nodes {S3, S6 and S8}.
- Next, find the set of all nodes which are reachable from {S3, S6 and S8} – and the answer is the set {S1, S2, S3, S5, S6, S8}.
- In the program dependency graph the nodes belonging in the slice have been identified in bold.

© Tripathy & Naik

6.3.2 Program Dependency Graph

Dynamic Slice

- A dynamic slice is more useful in localizing the defect than the static slice.
- Only one of the three assignment statements, S3, S6, or S8, may be executed for any input value of X.
- Consider the input value −1 for the variable X.
- For −1 as the value of X, only S3 is executed.
- Therefore, with respect to variable Y at S10, the dynamic slice will contain only {S1, S2 and S3}.

© Tripathy & Naik

6.3.2 Program Dependency Graph

Dynamic Slice

- For −1 as the values of X, if the value of Y is incorrect at S10, one can infer that either f_i is erroneous at S3 or the "if" condition at S2 is incorrect.
- A simple approach to obtaining dynamic program slices is explained here.
- Given a test and a PDG, let us represent the execution history of the program as a sequence of vertices $< v_1, v_2, ..., v_n >$.
- The execution history *hist* of a program P for a test case *test*, and a variable *var* is the set of all statements in *hist* whose execution had some effect on the value of *var* as observed at the end of the execution.

© Tripathy & Naik

6.3.2 Program Dependency Graph

Dynamic Slice

- Now, in our example discussed before, the static program slice with respect to variable Y at S10 for the code contains all the three statements – S3, S6, and S8.
- However, for a given test, one statement from the set {S3, S6, and S8} is executed.
- A simple way to finding dynamic slices is as follows:

  (i) for the current test, mark the executed nodes in the PDG.

  (ii) traverse the marked nodes in the graph.

© Tripathy & Naik

6.3.2 Program Dependency Graph

Dynamic Slice

- Figure 6.11 illustrates how a dynamic slice is obtained from the program with respect to variable Y at the end of execution.
- For the case X = −1, the executed nodes are: < S1, S2, S3, S4, S10, S11 >.
- Initially, all nodes are drawn with dotted lines.
- If a statement is executed, the corresponding node is made solid.
- Next, beginning at node S3, the graph is traversed only for solid nodes.
- Node S3 is selected because the variable Y is defined at node S3.

- All nodes encountered while traversing the graph are represented in bold.
- The desired dynamic program slice is represented by the set of bold nodes {S1, S2, S3}.

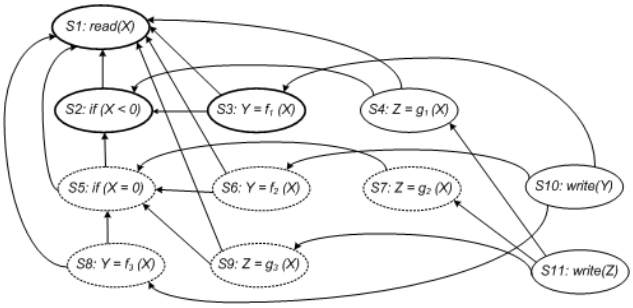


Figure 6.11 Dynamic program slice for the code in the figure 6.9, test case X= -1 with respect to a variable Y

© Tripathy & Naik

6.4 Ripple Effect

- Haney introduced the concept of ripple effect in the early 1970s.
- From the functional and performance perspectives, Yau, Collofello and McGregor defined the concept of ripple effect in 1978.
- They viewed ripple effect as a complexity measure to compare changes to source code.
- In the scheme of Yau et al., ripple effect is computed by means of error flow analysis.
- In error flow analysis, definitions of program variables involved in a change are considered to be potential sources of errors.

© Tripathy & Naik

6.4 Ripple Effect

- Inconsistency can propagate from those sources to other variables in the program.
- The other sources of errors are successively identified until error propagation is no more possible.
- This work was later extended to include stability measure.
- Stability reflects the resistance to the potential ripple effect which a program would have when it is changed.
- Stability analysis and impact analysis differ as follows:
  - stability analysis considers the total potential ripple effects rather than a specific ripple effect caused by a change.
- Next we explain how to compute ripple effect, based on the works of Sue Black.
- The basic idea is to identify the impact of a change to one variable on the program.

© Tripathy & Naik

6.4.1 Computing Ripple Effect

- Contained in module m1 of Figure 6.12, consider the three lines of code referring to variables a, b, and d.
- A change in the value of b will impact a in line (1), and it will propagate to a in line (2).
- Variable a affects variable d in line (2) and this will propagate to variable d in line (3).
- Based on the above example, intramodule change propagation is defined as the propagation of changes from one source code line in a module to another source code line within the same module.

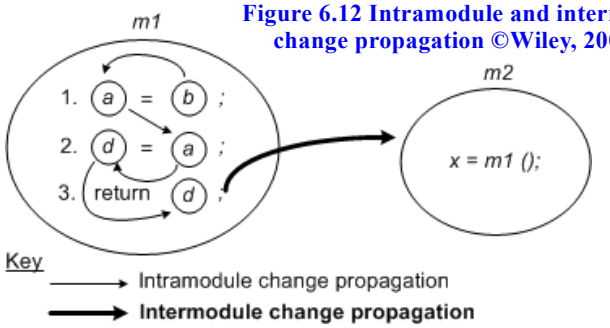


Figure 6.12 Intramodule and intermodule change propagation ©Wiley, 2001

© Tripathy & Naik

6.4.1 Computing Ripple Effect

- Propagation of values of variables in one module to variables in a different module is referred to as intermodule change propagation.
- Intermodule change propagation of values of a variable w occurs in the following ways:
  - If w is a global variable, then a change made to w by one module is seen by another module accessing w.
  - If w is an input parameter in a call to a second module, then values of w are propagated from the caller to the callee.
  - If w is an output parameter, then its value propagates from the module that makes an output to the module that accepts the output.
- Now we examine the code segment in Figure 6.12.
- Variable d propagates to any module that calls m1, because d appears in the return statement.
- If variable a is global, its appearance on the left-hand-side of an assignment statement causes its value to be propagated to any module that uses variable a.

© Tripathy & Naik

6.4.1 Computing Ripple Effect

- A matrix V_m is used to represent the initial starting points for intramodule change propagation.
- The matrix records the following five conditions of the module's variable x for all x:
  - x is defined in an assignment statement.
  - x is assigned a value in a read input statement.
  - x is an input to an invoked module.
  - x is an output from an invoked module.
  - x is a global variable.
- In V_m, variable definitions are uniquely identified.
- Variable occurrences satisfying any of the five conditions – defined, assigned, input, output, and global – are denoted by "1"; otherwise, an occurrence is denoted by "0".
- the notation x^d_i means that the variable x has been **defined** at line (i).
- Similarly, the notation x^u_i means that the variable x has been **used** at line (i).

© Tripathy & Naik

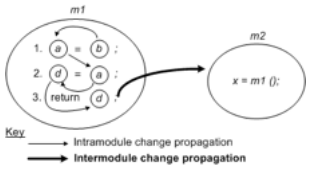
6.4.1 Computing Ripple Effect

- In module m_1, variable a is global and it is considered to be defined. Matrix V_{m1} for the lines of code in m_1 is expressed as:

$$V_{m1} = \begin{array}{c} \begin{matrix} a_1^d & b_1^u & d_2^d & a_2^u & d_3^u \end{matrix} \\ \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \end{pmatrix} \end{array}$$

- A zero-one (0−1) matrix Z_m indicates values of what variables propagate to other variables in the same module.
- Individual occurrences of variables are denoted by rows and columns of Z_m.
- The source code of module m_1 results in the following matrix:

$$Z_{m1} = \begin{array}{c} \\ a_1^d \\ b_1^u \\ d_2^d \\ a_2^u \\ d_3^u \end{array} \begin{array}{c} \begin{matrix} a_1^d & b_1^u & d_2^d & a_2^u & d_3^u \end{matrix} \\ \begin{pmatrix} 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \end{array}$$



- It is easy to observe that Z_{m1} is both reflexive and transitive.
- The reflexive property implies that every variable propagates to itself. whereas transitivity means that if v_1 propagates to v_2 and v_2 propagates to v_3 then v_1 also propagates to v_3.

© Tripathy & Naik

6.4.1 Computing Ripple Effect

- Suppose that module m1 is called by m3, a is a global variable, and m_2 and m_3 use a.
- If values of the variable corresponding to row i propagate to the module corresponding to column j, then the (i, j)th entry of the zero-one matrix is set to 1.
- For all the variables of a module m_1, propagation of their values to other modules is captured by an X matrix, denoted by X_{m1} as follows:

$$X_{m1} = \begin{array}{c} \\ a_1^d \\ b_1^u \\ d_2^d \\ a_2^u \\ d_3^u \end{array} \begin{array}{c} \begin{array}{ccc} m_1 & m_2 & m_3 \end{array} \\ \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \end{array}$$

The intermodule change propagation for variables occurring in m_1 is obtained by means of the Boolean product of the two matrices Z_{m1} and X_{m1}, as follows:

$$Z_{m1}X_{m1} = \begin{pmatrix} 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

In the Boolean product $Z_{m1}X_{m1}$, the "1" in row 2, column 3 indicates change propagation from b_1^u to m_3; similarly, the "0" in row 3, column 2 indicates no change propagation from d_2^d to m_2.

© Tripathy & Naik

6.4.1 Computing Ripple Effect

The Boolean product of V_{m1} and $Z_{m1}X_{m1}$ indicates the variable definitions that propagate from m_1 to other modules:

$$V_{m1}Z_{m1}X_{m1} = \begin{pmatrix}1 & 0 & 1 & 1 & 0\end{pmatrix}\begin{pmatrix}0 & 1 & 1\\ 0 & 0 & 0\\ 0 & 0 & 0\\ 0 & 0 & 0\\ 0 & 0 & 1\end{pmatrix} = \begin{pmatrix}0 & 1 & 3\end{pmatrix}$$

Now, $V_{m1}Z_{m1}X_{m1}$ indicates that there are no change propagations to m_1, one change propagation to m_2, and three change propagations to m_3.

- Concerning the complexity of making changes, the more complex a module is, the more resources are needed to change the module.
- Therefore, a measure of complexity can be factored into the calculation of change propagation to obtain a measure of the complexity of modifying the definitions of variables.
- ~~The well-known McCabe's cyclomatic complexity can be integrated with the ongoing computation of change propagation.~~

© Tripathy & Naik

Summary

General Idea

Impact Analysis Process

Dependency-based Impact Analysis

Ripple Effect

Change Propagation Model

© Tripathy & Naik